

综合练习：无 IAT 版注入器雏形

● 本节目标：

- 整合前六章节的知识，编写一个完整的注入器。
- 全程不使用 GetModuleHandle、GetProcAddress 等敏感 API。
- 所有 API 地址通过 GetProcAddressR 动态解析。
- 注入一段极简 Shellcode (mov eax, 42; ret)，验证注入成功。
- 用 IDA 检查 IAT，确认无敏感函数名泄漏。

● 程序整体结构

注入器主流程：

- ① 通过 PEB 遍历 + 哈希，拿到 kernel32.dll 基址
- ② 通过 GetProcAddressR，动态解析：
 - OpenProcess
 - VirtualAllocEx
 - WriteProcessMemory
 - CreateRemoteThread
 - WaitForSingleObject
 - CloseHandle
- ③ 用 OpenProcess 打开目标进程
- ④ 用 VirtualAllocEx 在目标进程申请内存
- ⑤ 用 WriteProcessMemory 写入 Shellcode
- ⑥ 用 CreateRemoteThread 执行 Shellcode
- ⑦ 等待线程结束，清理资源
- ⑧ 用 IDA 检查 IAT，验证隐蔽效果

● Shellcode 设计

本节使用一段极简的 Shellcode，功能是“把 42 放进 EAX 寄存器，然后返回”。这段代码不产生任何可见效果，但可以验证注入流程是否跑通。

```
// x64 Shellcode: mov eax, 42; ret
unsigned char shellcode[] = {
    0xB8, 0x2A, 0x00, 0x00, 0x00,    // mov eax, 42
    0xC3                             // ret
};
```

● 完整代码实现

(一) 把以前封装好的 .h 文件复制进来（注：代码见目录）

(二) 预计算哈希值

用第 5 节的方法，离线算出所有目标函数的哈希值：

```
// 这些数字需要自己用 Ror13hash 提前算好，然后硬编码
define HASH_KERNEL32                /* 这里为 Hash 值 */
define HASH_OPENPROCESS              /* 同上 */
define HASH_VIRTUALALLOCCEX         /* 同上 */
define HASH_WRITEPROCESSMEMORY      /* 同上 */
define HASH_CREATEREMOTETHREAD      /* 同上 */
define HASH_WAITFORSINGLEOBJECT      /* 同上 */
define HASH_CLOSEHANDLE              /* 同上 */
```

计算工具

```
int main() {
    const char* targets[] = {
        "kernel32.dll",
        "OpenProcess",
        "VirtualAllocEx",
        "WriteProcessMemory",
        "CreateRemoteThread",
        "WaitForSingleObject",
        "CloseHandle",
    };
    for (auto name : targets) {
        std::cout << name << " = " << std::dec << Ror13hash(name) << std::endl;
    }
}
```

(三) 主注入流程（注：代码见目录）

● 验证 IAT 隐藏效果

编译完成后，在 IDA 中打开生成的 EXE：

- Imports 窗口（View → Open Subviews → Imports）：应该只有 ExitProcess 和少数系统调用，不应该出现 VirtualAllocEx、WriteProcessMemory、CreateRemoteThread、OpenProcess。
- Strings 窗口（Shift + F12）：搜索 CreateRemoteThread、VirtualAllocEx 等等，都应该搜不到。
- 如果搜到了：说明代码里某处还有明文。检查是否有 define HASH_XXX 里带着注释明文，或者是否有未删除的验证代码。

● 可能的坑

坑	现象	解决
哈希值硬编码错误	GetProcAddressR 返回 nullptr	用离线工具重新算
PROCESS_ALL_ACCESS 权限不足	OpenProcess 失败	以管理员身份运行
目标进程是 32 位	注入后崩溃	本节暂只支持 64 位目标
Shellcode 写错	目标进程崩溃	确认 Shellcode 字节序和指令正确
IDA 中看到明文	有残留字符串	删掉所有验证代码，只保留硬编码哈希